

Contextualizing Data-Sparse QSPR Property Prediction with Categorical Representations Built from Variational Autoencoders

Johnathan Ahdout

John L. Miller Great Neck North High School, Great Neck, NY

In Affiliation with Brown Lab at Vanderbilt University's

Department of Pharmacology

Mentor: Dr. Benjamin Brown

Abstract

Small molecules interact with proteins to prevent disease-related mutations. Computational aided drug design uses computers to screen molecular libraries for bioactivity. However, many molecules fail synthesis due to metabolic instability or unsuitability for human consumption. A tool to predict QSPR metabolic stability before synthesis could save time and money in research and development, but no such tool exists due to sparse data on universal metrics like hepatocyte intrinsic clearance (Esaki et al., 2019). This study addressed this gap by contextualizing metabolic stability using variational-autoencoder-generated (VAE-generated) categorical stability representations for QSPR property prediction of metabolic stability. A VAE was trained on the structural data of 2.4-million molecules' drug-like Morgan fingerprints to embed relationships between structure and activity of drug-like molecules in a Gaussian latent space. Copies of the VAE were fine-tuned on molecules categorized as stable or unstable to provide context to QSPR models and improve stability prediction. These fine-tuned models generated categorical representations (CRs) of stability, which were combined with Morgan fingerprints of new samples for a QSPR metabolic stability binary predictor. The control group achieved 87% prediction accuracy, while the experimental group, with contextualized data, achieved 85%, showing no significant improvement over state-of-the-art models in this proof-of-concept state. Despite limitations such as the simplicity of Morgan fingerprints and potential data overlap, this model demonstrates potential to reduce costs and accelerate drug design. Future research will explore data augmentation with CRs and the use of SMILES BERT embeddings with Kolmogorov-Arnold Network layers to improve data extraction.

Introduction

Overview of Research Problem

Computational-Aided Drug Discovery (CADD) Methods

Metabolic stability is a pivotal determinant in drug development, critically influencing the pharmacokinetics and pharmacodynamics of pharmaceutical agents. It governs the duration a drug remains bioactive before undergoing metabolic degradation, thereby impacting bioavailability, half-life, and adverse effect profiles (Gunaydin et al., 2018). Empirical studies indicate that approximately 40% of drug candidates are terminated during clinical trials due to suboptimal metabolic stability, underscoring the necessity for precise early-stage prediction methodologies (Smith & Jones, 2011). Computational techniques, particularly Quantitative Structure-Activity Relationship (QSAR) and Quantitative Structure-Property Relationship (QSPR) modeling, have become indispensable, offering substantial reductions in time and cost compared to traditional *in vitro* and *in vivo* methods (Johnson, 2013). Research endeavors have been focused on producing adequate QSPR models to narrow down molecular libraries based on metabolic stability to save money and time by only synthesizing stable candidates, but there are a number of limitations that have limited the development of such models.

Inadequacies in Molecular Representation Quality, Availability, and Predictive Generalizability

Despite their utility, structure-activity and structure-property relationship modeling are limited by the exhaustive search required in virtual high-throughput screening (vHTS) (Subramaniam et al., 2008). The expansive possibilities for chemical structures of drug-like molecules and reliance on fixed libraries have led to inefficiencies and incomplete exploration of potential drug-like molecules. Furthermore, conventional molecular representations such as molecular fingerprints frequently fail to capture the intricate structures of molecules, leading to data sparsity and suboptimal model accuracy (Cherkasov et al., 2014). As such, researchers have invested in Simplified Molecular Input Line Entry System (SMILES) or graph-based representations to facilitate the encoding of molecular structures into text strings or three-dimensional conformations of molecules in a higher spatial representation respectively (Weininger, 1988). While these methods enable easier manipulation and processing, these representations fail to be rapidly scalable to larger models and require more computational power than usually available. Therefore, researchers tend to default to less data-rich, but more easily scalable representations such as Morgan fingerprints, binary vectors representing the presence of a number of preset substructures of a molecule, to train models for industry. This further complicates predictive modeling by introducing pertinent tradeoffs that affect usability in industry, so researchers require methods to extract diverse data on massive scales and context from limited representations (Gasteiger & Engel, 2003).

Additionally, predicting metabolic stability is more complex than predicting the activity of small molecules against a specific target (Kerns & Di, 2008). Unlike target-based activity prediction, which benefits from the defined structural context of a binding pocket, metabolic stability is treated as an intrinsic property of the small molecules themselves due to the usual profile of targets encountered within standard metabolic pathways. This intrinsic nature, coupled with the absence of a target to narrow the search space, poses a significant hurdle in prediction, as the variability in metabolic stability arises from subtle and often unpredictable molecular features. Researchers often default to a general measure of

stability such as hepatocyte intrinsic clearance, or the ability of the liver to metabolize a drug (Esaki et al., 2019). This metabolic stability representation is suitable for classification, but has less than ten thousand data points for model training, testing, and validation, limiting scalability.

Optimization of Data-Sparse CADD

The scarcity of high-quality, annotated datasets exacerbates the challenge of predicting metabolic stability. Traditional machine learning algorithms implemented within QSPR models have often underperformed when faced with such sparse data, leading to suboptimal predictive outcomes (Medina-Franco, 2021). In response, transfer learning has emerged as a crucial technique to address this limitation. By leveraging pre-trained models on large datasets, transfer learning enables the fine-tuning of these models on more specific, limited datasets, thereby enhancing predictive accuracy (Pan and Yang, 2010). In computational chemistry and drug design, this approach facilitates the development of robust predictive models by utilizing extensive pre-existing molecular data and refining the model with targeted data. This methodology is particularly advantageous in drug discovery, where generating experimental data is both costly and time-consuming (Chen et al., 2012).

Applications in Computational-Aided Drug Discovery

Researchers' attempts to extract molecular significance from structures with QSPR models have been limited by their ability to consistently predict molecular properties from available data in the context of identifying candidates for small molecule drug design. For instance, a major problem in dopaminergic receptor antagonist identification for development of an adjuvant therapy for Parkinson's Disease is the difficulty to pin down the binding mode of antagonists based on chirality, potentially leading to inaccurate QSAR models (Jones et al., 2024). Binding modes have been long sought after through more computational validation, but to no avail due to sparse data. This opens the door to models that can make best use of sparse data of antagonists to determine the stability of such molecules and their activities in particular binding modes to verify QSAR models.

Transfer and Few-Shot Learning for Molecular Property Prediction

Few-shot methods attempting to predict chemical properties, particularly binding affinity to particular pockets, have been used, such as that of Eckmann et al. (2024), which utilized a context encoder module to associate activity values and structures to be fed into a predictor network. However, such methods showed insignificant results and reported a slew of limitations such as the data sparsity of representations like Morgan fingerprints and rudimentary context aggregation yielding little to no improvement.

Such transfer learning approaches explored a plethora of methods, but have infrequently explored the problem of metabolic stability and yielded promising results. Torres et al. (2023) attempted molecular property prediction with graph neural networks and transformers, which had the advantage of geometric atom-bond featurization in the form of nodes and edges, as well as utilizing tokenization of functional groups through multi-head self-attention. This method's transfer learning module implemented with meta-learning modules was unable to consistently extract relevant information from graph structures with significantly lower computational power, which demonstrates the pertinence of few-shot learning modules' weight in the development of such methods to be reliably used in industry. As such, methods

leveraging sufficiently relevant and data-rich context as well as few-shot learning prioritization with learnable data under computational power constraints are important to develop.

Variational Autoencoders (VAEs) & Molecular Representation Quality

Among model architectures posited to improve molecular property prediction, variational autoencoders are vastly underutilized and have been written off as being unsuitable for uncovering structure-property relationships. According to Muenkler et al. (2022), VAEs have demonstrated insufficient models for reconstructing molecular graphs based on dimensionality reduction and feature extraction, but excel at latent space optimization of information. Despite this relatively discouraging report, it can be seen that graph representations for small molecules are, on average, outperformed by descriptor-based (e.g. SMILES strings) models. This was consistent for predictions of a variety of molecular properties in terms of predictive accuracy and computational efficiency, so descriptor-based representations are promising to leverage the latent space optimization properties of VAEs (Jiang et al. 2021).

Variational autoencoders have also shown immense promise for molecular structure and feature generation, as the advent of dimensionality reduction and data diversification in the form of the distribution-based latent space prove essential for high-throughput screening methods (Simonovsky & Komodakis, 2018). Additionally, this statistical component could be valuable in representing complex molecular structures, which has proven successful in constructing chemical latent space (Figure 1) from large-sized compounds that were unable to be handled by existing methods (Ochiai et al., 2023). This advantage in structural complexity and diverse featurization of chemical space points to the potential application of VAEs in molecular property prediction with sparse, yet complex data that can be represented in terms of expansive feature spaces available for contextualization.

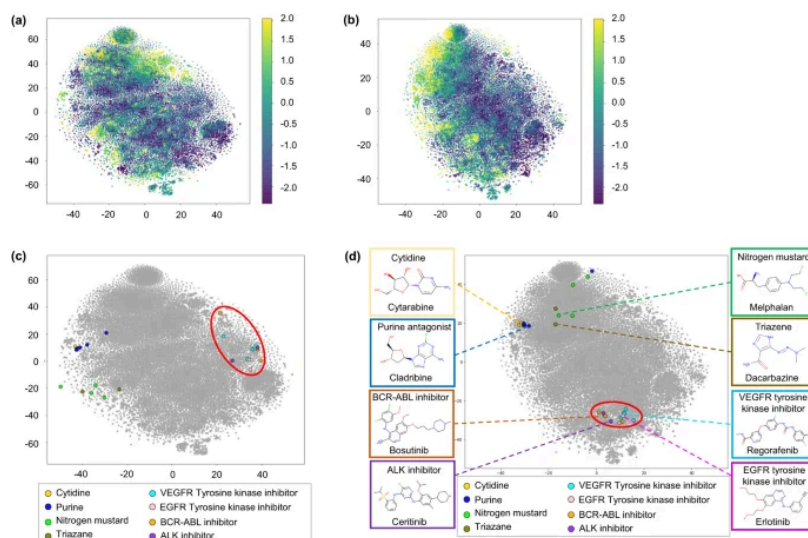


Figure 1 shows the t-SNE visualization of the chemical latent space trained by Ochiai and colleagues. The space clusters similar substructures and separates complex functional groups as seen in the bottom right of the image. Such specificity in structural representation with a probabilistic component can lead to generalization in diverse dimensionality amplification and context production for training. The clustering of similar substructures also shows potential for transfer learning of such clusters with sparse data. Figure by Ochiai et al. 2023.

Latent Space Categorical Molecular Representations

Embeddings of VAEs lend themselves to data-rich low dimensional representations with potential for generalization in dataset augmentation or generation, and have been employed to pre-compress data to encourage classification accuracy (Lovrić et al., 2021). Discrete representations of molecules have been enriched by the dimensionality amplification and reduction techniques akin to VAEs, demonstrating the potential of continuous latent vectors (Figure 2) to become strategic tools for contextualization in few-shot learning by means of previous success in diverse and complex molecule generation (Gómez-Bombarelli et al., 2018). These successes with VAEs present a novel conception of categorical representations (CRs) of molecular stability levels being fine tuned with sparse data and amplified by rich latent space probabilistic representations to be used as context for a prediction module in a few-shot learning approach to molecular property prediction.

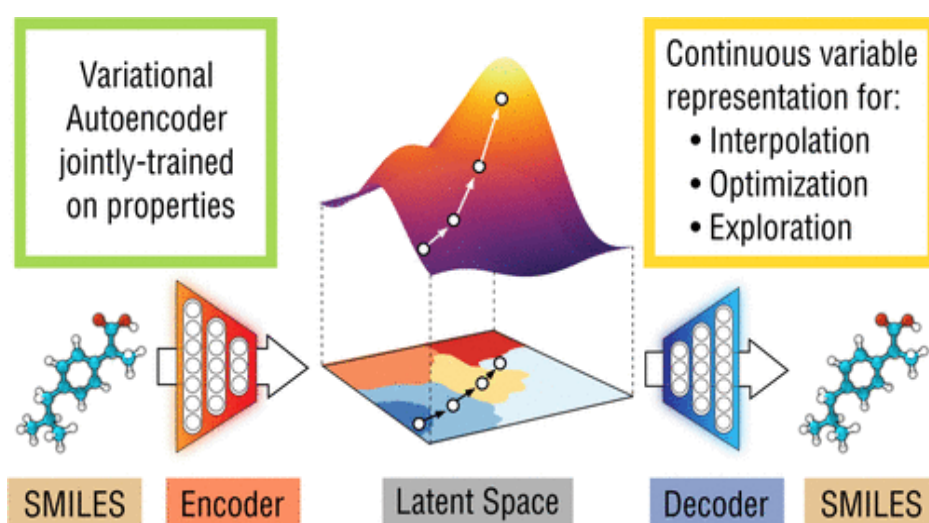


Figure 2 shows the graphical abstract of Gómez-Bombarelli and colleagues demonstrating the encoding of rich latent spaces for molecular property prediction, which has contributed to the formulation of a novel method for categorical representations of metabolic stability in fine-tuned models serving as data-rich context for prediction. Figure by Gómez-Bombarelli et al. 2018.

Problem Statements & Objectives

1. Molecular representations often struggle to robustly define structural features when data is sparse. This is due to the large computational power needed to lower dimensionality, assess similarities, correlate metabolically relevant properties between query molecules.
 - ◆ **Objective 1:** Pre-train a variational autoencoder with Multilayer Perceptrons (MLPs) for encoder-decoder modules to learn a latent space of structures of various drug-like compounds based on SMILES strings. Novel representations of certain molecular structures in correlation to one another automatically clustered by similarity in the latent space allows for future predictors to produce more consistent results with the context of other similar molecules within a category.

2. Current models have trouble distinguishing stable and unstable molecules from one another based on structure due to sparse data and inherent challenge in predicting properties of solely small molecules with complex and difficult to describe structures.
 - ◆ **Objective 2:** Fine-tune the autoencoder in 3 distinct and separate versions where we few-shot train on CRs of metabolic stability levels: stable, moderate, or unstable molecules. This will allow the transformation of the latent space to create a better distinction between the two.
3. Sparse molecular data lacks context for molecular property prediction modules to be properly generalizable to new chemical space for use in industry.
 - ◆ **Objective 3a:** Create a predictor model to predict the stability of the molecules, where input to the model will be a concatenated vector of the sample/query molecule, and a stable, moderate, and unstable molecule augmented by their respective fine tuned CR models. This will essentially be processing the latent layers of the general module, stable CR module, moderate CR module, and unstable CR module altogether. This will give the model sufficient and regeneratable context on the stability of these molecules, which will be crucial due to the sparse data available, so it can train more efficiently and get the best results possible out of the data presented.
 - ◆ **Objective 3b:** Compare the performance of a model trained on CR-contextualized data and non-CR data to determine the significance of improvement of QSPR modeling with data contextualization through categorical representation. Create a prediction model accessible and customizable through the command-line interface to a program simplifying machine learning processes for computational aided drug design.

Methodology

Research was conducted at the Brown Lab at Vanderbilt University's Center for Applied AI in Protein Dynamics from July 3rd, 2024, to August 24th, 2024. Construction of model architecture, as well as training, testing, and command-line interface hyperparameter fine-tuning, was done exclusively by the student researcher over the course of an eight-week internship at the lab. Mentorship from Dr. Benjamin P. Brown was provided along the way, particularly to complete the VAE construction step of the methods. The proprietary CADDCore cheminformatics PyTorch toolkit developed by Dr. Brown was utilized as a scaffold for model procedures and training hyperparameter customization from the command-line interface. All procedures were integrated into CADDCore.

Equipment and Materials

Computations were performed on a Dell Precision 5860 tower workstation powered by Intel Xeon W5-2465X (16 cores/32 threads), 128 GB RAM, and an NVIDIA RTX 4500 Ada GPU. 2.4 million molecules in SMILES format were sourced from the ChemBL online database and used for VAE training, while DMPK data and SMILES strings of molecules of certain metabolic stability levels were sourced from a publication in *Molecular Informatics* by Esaki et al. in 2019.

Model Architecture and Context Augmentation

Step 1: VAE Training and Architecture

A variational autoencoder was integrated into CADDCore to take the ChemBL molecules as input, convert them into Morgan fingerprint embeddings, and reconstruct them in an attempt to produce a scaffold, or format of structural features for chemical structures to learn, for context generation (Figure 3, Appendix A).

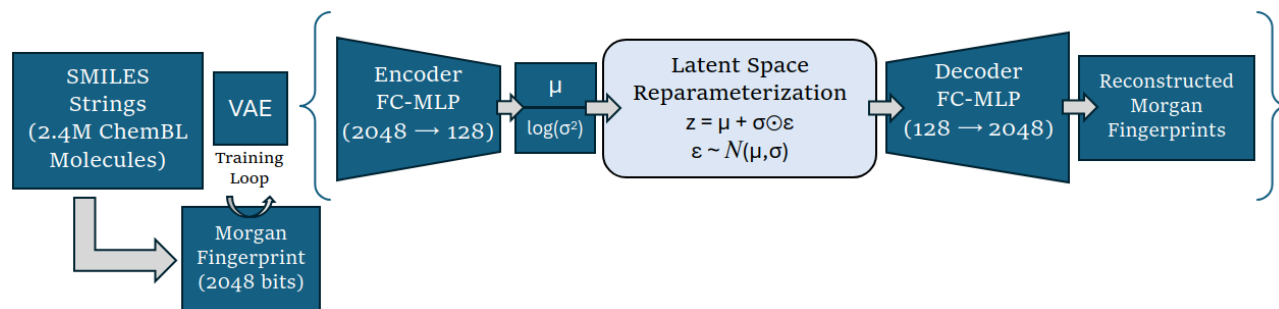


Figure 3. Schematic diagram of VAE Architecture and training loop. This was designed to embed a relevant chemical space for context generation of stable, moderate, and unstable molecular features for metabolic stability prediction tasks. Created by student researcher.

The CADDCore Morgan fingerprint conversion was done with RDKit's Chem package and then fed into a variational autoencoder in order to embed molecular representations across a broad range of chemical space for context generation capability. Multilayer perceptron encoder modules with 3 fully connected hidden layers of sizes 1024, 512, and 256 reduced the dimensionality of Morgan fingerprints from size 2048 bit vectors to 128 mean and log-variance metrics. 10% input dropout for the encoder was employed to prevent overtraining and sufficient variety in generation for a vast contextual chemical space. Log-variance was implemented as it is the standard for variational autoencoders instead of standard deviation due to the increased precision of such a variably scaled metric.

A reparameterization trick standard for modern VAEs was employed in the latent space to ensure the stochastic backpropagation through and mean and log-variance outputs of the encoder. In particular, this reparameterization trick was able to ensure learned standard deviation properties from a random-sampled component of the learning process, which would usually hinder the ability to backpropagate on weights due to randomness being untunable by derivatives in a standard VAE. This was achieved by extracting standard deviations from the log-variance tensors and drawing a random sample from a normal distribution with each corresponding standard deviation. The output of the latent spaces was then computed as a sum of means and the product of the random sample and the corresponding standard deviation. A script to document the evolving latent space distribution through the training loop with Matplotlib was produced (Figure 4, Appendix B).

A fully connected MLP decoder module was constructed to expand the dimensions of the features embedded in the latent space back to 2048. A reconstruction loss of mean-squared error and regularization loss of standard Gaussian Kullback-Leibler divergence were weighted equally to minimize loss throughout the training loop. KL Divergence was utilized to minimize the divergence of the constructed latent space distribution from a standard Gaussian, pushing the chemical space representation

towards regular sampling capabilities. After training the VAE for 100 epochs with a 0.001 learning rate, the Adam optimizer, a mini-batch size of 4096, and running a train-validate split of 9:1 for the 2.4 million Morgan fingerprints, the training convergence shown in Figure 5 indicated significant loss minimization and molecular reconstruction that could be suitable to move onto the fine-tuning step.

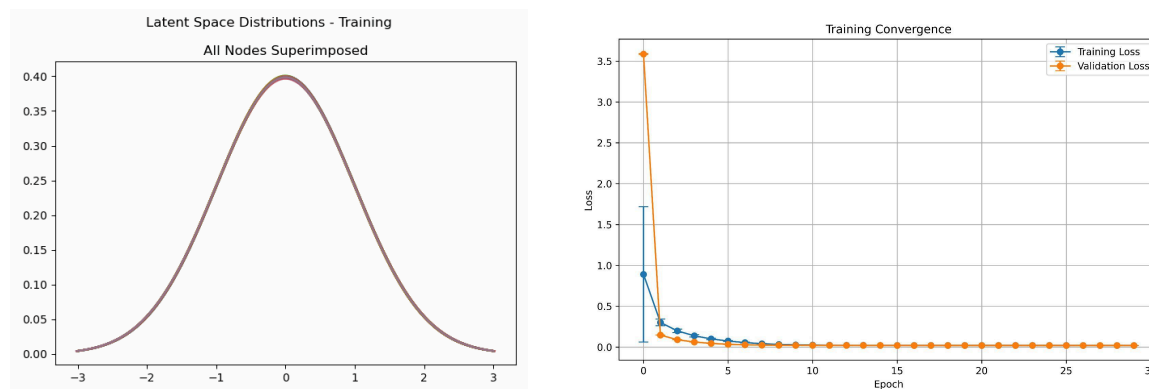


Figure 4 (left) depicts the distribution of mean and standard deviations converged upon as training proceeds in the latent space. Converging on a standard Gaussian distribution signifies that the context generation with the fine tuned VAEs will be able to generate reliably from such a standard Gaussian for ease of training, rather than having to sample from an evolving nonstandard distribution that is not readily available or static during the decoding process. **Figure 5 (right)** depicts training convergence of the original VAE with the ChemBL SMILES strings. Created by student researcher.

Step 2: Fine-Tuning with Transfer Learning

In order to generate context of stable or unstable molecules sufficient for metabolic stability prediction, the original VAE was finetuned three separate times on sets of SMILES strings with varying metabolic stability levels: stable, moderately stable, and unstable. SMILES datasets were acquired from a paper by Esaki et al. in 2018, and each set was split in half to avoid data leakage or overtraining. One half of the set was used to fine tune the VAE for CR generation, while the other half was saved for the prediction training and testing (Figure 6).

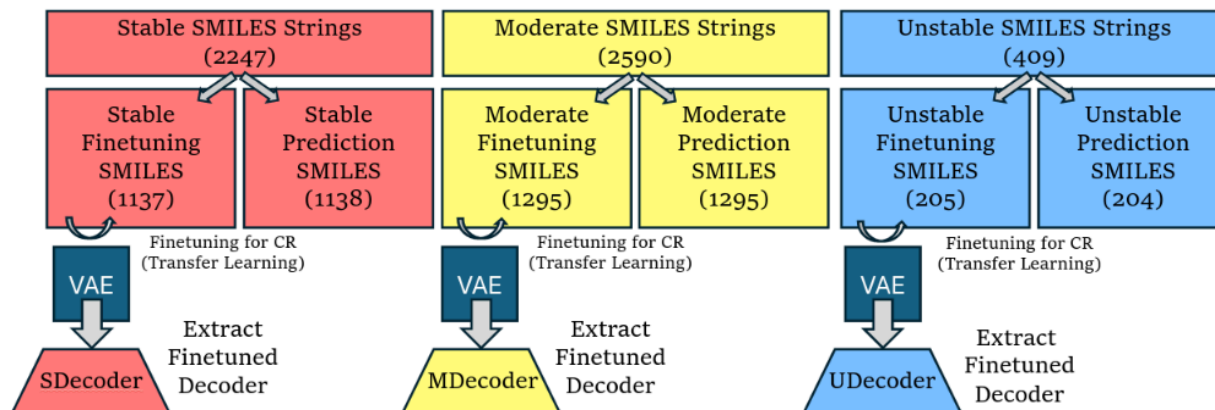


Figure 6. A schematic for the transfer learning step, showing the data split to prevent data leakage, fine-tuning three VAEs separately, and extracting the fine tuned decoder for CR generation in the next step. Created by student researcher.

The trained VAE model was loaded each time with the CADDCore command-line run_protocols script. Each loaded model was transferred for training further for 20 epochs with the same aforementioned hyperparameters, save the improved 7:3 weighting for reconstruction to regularization loss, which is visualized as training convergence and latent space visuals for each fine tuned model (Figure 7). The specific train-test split was decided upon through parallel processing binomial splits of varying proportions, and converging on the split that most reliably minimized loss across adequate trials. The decoders of each of these fine tuned models were extracted to generate categorical representations in the next step of the methods.

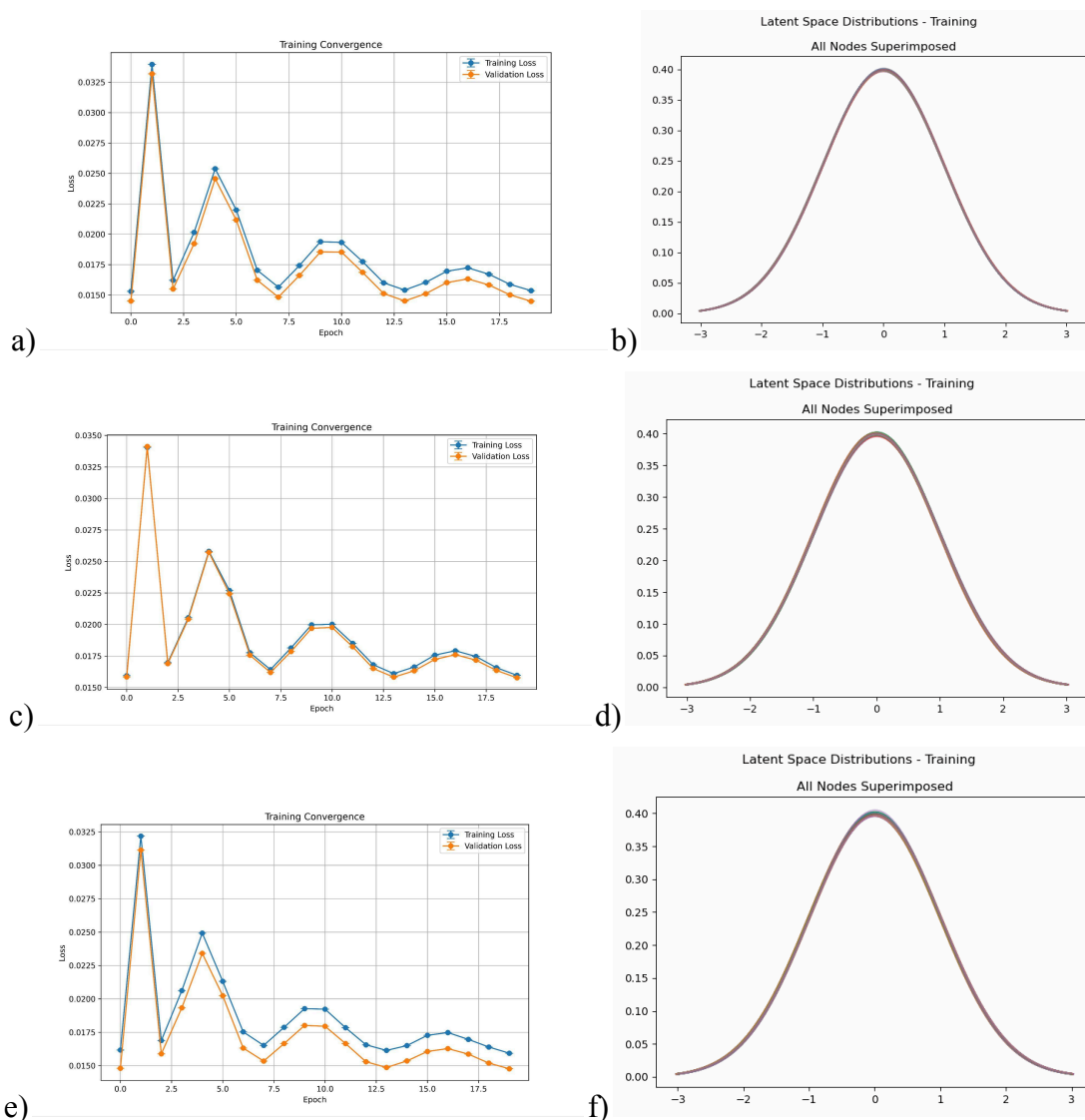


Figure 7. Shows training convergence of stable (a), moderate (c), and unstable (e) fine tuned VAEs, and latent spaces of stable (b), moderate (d), and unstable (f) fine tuned VAEs. Created by student researcher.

Step 3: Categorical Representation (CR) Context Generation

For each metabolic stability level and fine tuned VAE's extracted decoder, categorical representations (CRs) were generated to be concatenated as context for the prediction task. 128 random samples from a standard Gaussian were pushed through the decoder 2637 times (the number of samples

from the sum of the prediction half of the Esaki SMILES), which can be done as every latent space visualization converged to a standard Gaussian. This generation was done to produce Morgan fingerprint embeddings with 2048 features each for CR molecules for every metabolic stability level (Figure 8). These were concatenated every generation loop to produce a tensor of CR context molecules of size 2637 by 2048 (one Morgan fingerprint per sample out of the 2637 total).

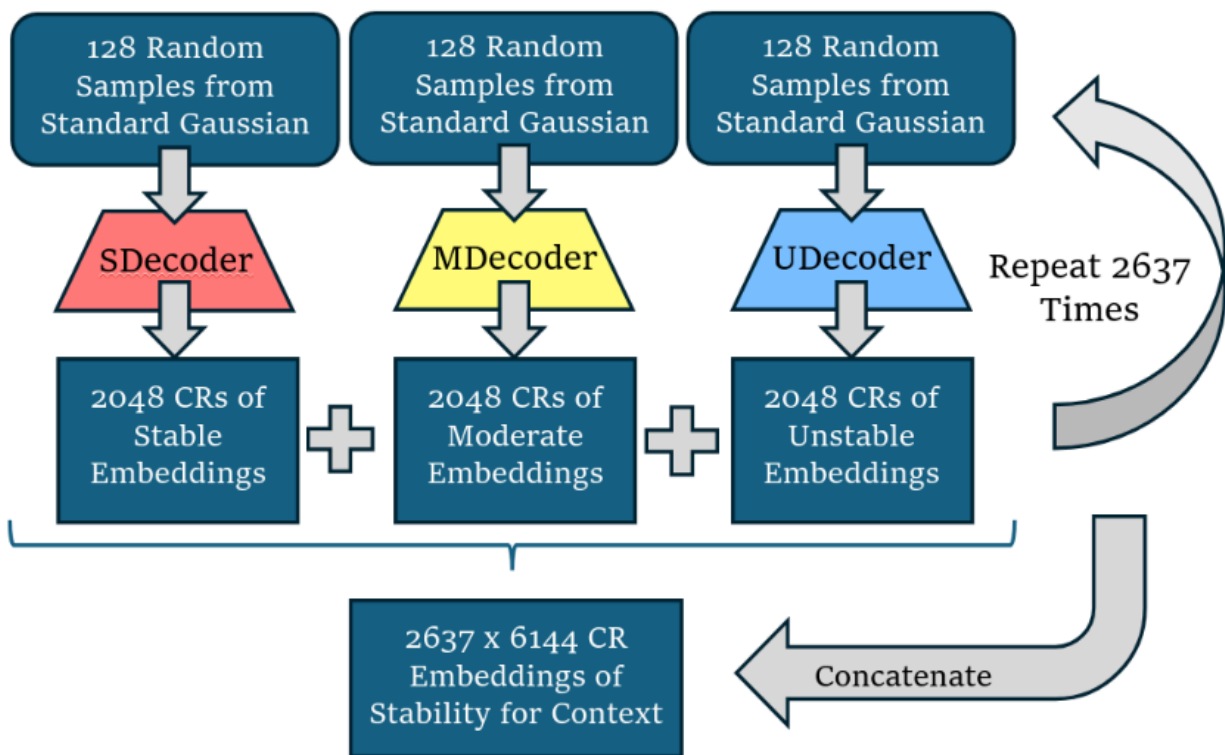


Figure 8. Schematic of CR generation step of methods in order to mimic reconstruction of stable, moderate, and unstable Morgan fingerprint samples. Created by student researcher.

Step 4: Data Processing and Metabolic Stability QSPR Model Architecture

The generate_dataset script within CADDCore was used to convert the prediction split SMILES strings mentioned in step 2 into Morgan fingerprints. These were concatenated with the generated CRs and binary result labels for the appropriate classification. Result labels of 1 were concatenated to the stable set, while moderate and unstable fingerprints were assigned 0s to differentiate between stable and moderate molecules for the most precise classification. The prediction molecules were also concatenated with their respective result labels without the CRs to test training a prediction module without CRs as a baseline for performance (Figure 9).

A fully connected MLP with a hidden layer of size 32 and 25% dropout was constructed to classify the 2637 molecules with context as stable (1) or unstable (0). The datasets produced in step 4 was split in a 6:1:1 train:validate:test split, and ran through individual MLPs for 100 epochs, with MSE loss functions, stochastic gradient descent optimization with 0.9 momentum, a mini-batch size of 32, 5% input dropout, a sigmoid activation function for each layer as is the classification standard, and a 0.0005 learning rate. Classifications were made for the testing set, and the models were compared by AUC,

logAUC, F1 score, MCC score, and AUC parity scores for performance. Steps 3 through 5 can be seen in the script in Appendix C.

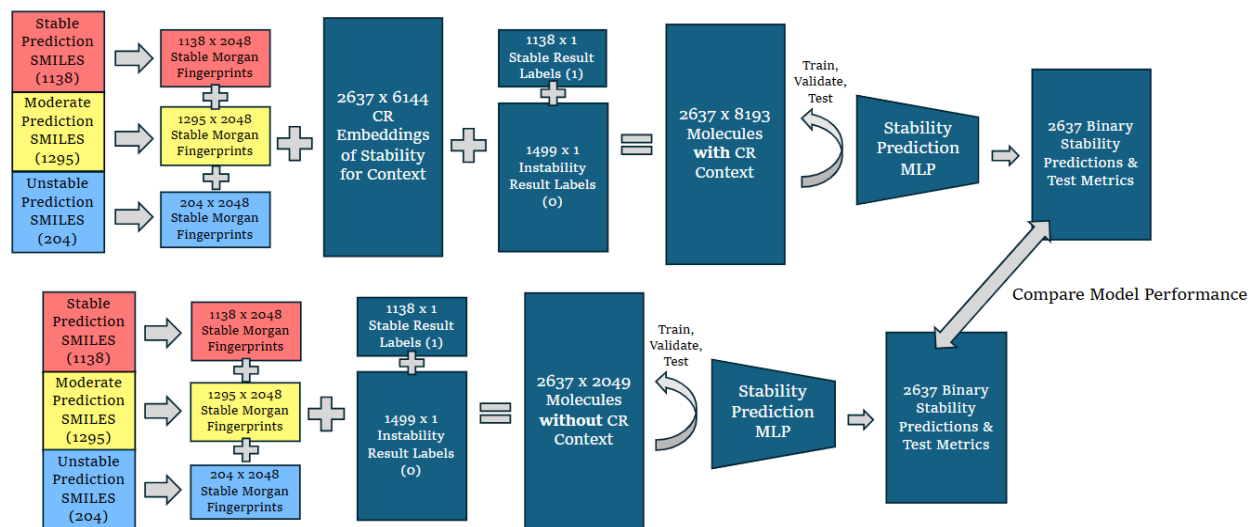


Figure 9. Step 2's split of prediction SMILES were converted to Morgan fingerprints, concatenated vertically, and concatenated horizontally with respective result labels, both with and without CRs. Data is fed into a fully-connected MLP QSPR metabolic stability binary classification model. Created by student researcher.

Results

Matplotlib and Dr. Brown's proprietary CADDCore cheminformatics toolkit were used to plot the Receiver-Operating Characteristic curves of the CR trained and non-CR trained models and collect the ROC-AUC metrics to measure the performance of each model (Figure 10).

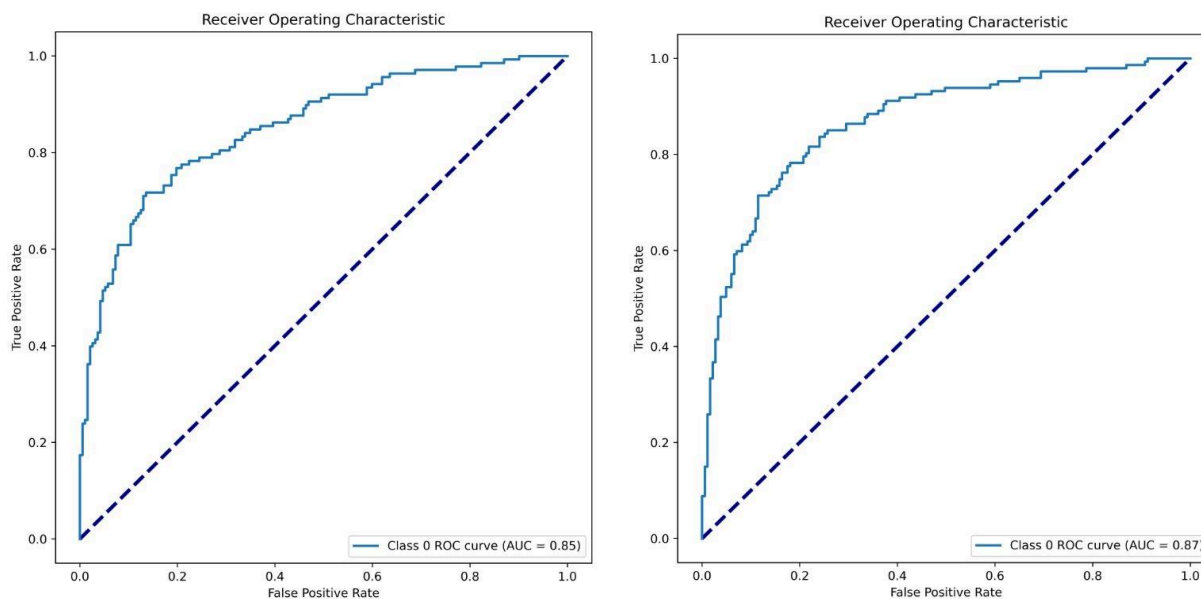


Figure 10. The ROC curve (TPR vs. FPR) for both the CR trained model (left) and non-CR model (right). The CR model had an AUC score of 0.85, while the non-CR model had an AUC score of 0.87. Created by student researcher.

Along with the AUC metrics for each model, the Matthew's correlation coefficient (MCC) to indicate the quality of binary classification (stable vs. unstable) and F1 score to measure precision and recall were used to delineate the performance of each model (Table 1).

Table 1. The below table indicates the *MCC and *F1 scores for each model's testing. Created by student researcher.

	CR trained model (experimental)	Non-CR trained model (control)
ROC-AUC	0.8522	0.8686
MCC	0.5559	0.5937
F1	0.7380	0.7692

Discussion

The model performance, classification capacity, and precision-recall metrics in ROC-AUC, MCC, and F1 scores were not improved by the VAE generated CR training, but there was also no significant decrement in QSPR prediction. In any case, each models' respective simplicity limited their performance to below the conventionally accepted threshold of successful QSPR models at an ROC-AUC of 0.9 (Rakhimbekova et al., 2020). The lack of improved accuracy over the state-of-the-art in the form of ROC-AUC scores hovering around 0.9 can be attributed to the various limitations of this approach in its early stages of development without optimization.

Limitations

A multitude of limitations present in this study necessitate further research and improvement on this method, as VAE-generated categorical representations may still hold promise if implemented more robustly. Firstly, Morgan fingerprints being used as the molecular structure data representation limited the data richness encoded by categorical representations due to their binary nature, so supplying a more data-rich representation, such as graph-based representations for message-passing neural networks, Extended-Connectivity Fingerprints, or BERT SMILES embeddings, for an implementation beyond proof-of-concept is necessary. Additionally, the simplicity of the binary classification of stable and unstable molecules by way of hepatocyte intrinsic clearance thresholds limited the complexity of QSPR prediction, so a more robust or multiclass classification is in order. The sizes of the datasets for the non-CR and CR trained models were also not exactly the same, which could have slightly affected the F1 scores for each model as they are most effective measures with equally sized data sets up for comparison. Finally, the 2.4 million drug-like molecules that the general VAEs trained on were not screened for overlapping molecules with the stable, moderate, and unstable molecules sourced from Esaki et al., which may have caused unnoticeable yet significant data leakage.

Future Research

This project is its first stage, and has multiple further complexities to be addressed to produce a robust product for use in industry. Firstly, out-of-distribution testing can be improved through leave-class-out validation on the QSPR set, which can achieve more precise classifications for data with more noise.

Additionally, Morgan fingerprints' data simplicity can be accounted for by swapping the data representation to BERT SMILES string embeddings, which can more accurately embed the local and global information of a small molecule while retaining the contextual information of certain problems they are applied to for easier integration into simpler deep-learning frameworks. In particular, BERT SMILES embeddings show promise in communicating tautomerism and chirality, among other global properties of drug-like molecules in comparison to one another. Graph-based structures and Extended-Connectivity Fingerprints may also be promising alternatives that require more computational power, but may be worth the tradeoffs.

Data augmentation of sparse data sets can also be tested with the variational autoencoder approach to categorical representation context generation, as combinations of previously seen data can be combined with new CRs to reuse data without overtraining. New contextual representations can allow the model to refine its prediction of a sample it has seen multiple times without correlating their outputs with one another, which can allow the model to make better use of its sparse samples to generalize its weights and biases for out-of-distribution testing. Bias and overtraining can be avoided through oversampling and leave-class-out cross validation, as previously mentioned. This can be implemented to not only retain more specific information about particular data to increase the complexity of the prediction, but can also remedy the data sparsity problem if successful in combination with contextualization as implemented by the objectives of this study.

Future models can also integrate a statistical component to the prediction to produce a distribution of molecules from stable to unstable in order to indicate the distance of an inputted molecule to a more stable version by comparing to CRs of similar structures, potentially through a neighborhood aggregative graph neural network approach. KL Divergence metrics can be measured through losses akin to variational autoencoder latent spaces in order to visualize differences between distributions for a given molecule's training loop, therefore differentiating between stable and unstable molecules and quantifying distance from another class' output. The capacity of a QSPR model to not only predict the metabolic stability of a given molecule but also to find a stable molecule with similar properties can be extremely useful in industrial applications to maximize stability without sacrificing potentially already optimized bioactivity.

The complexity of QSPR prediction modules similar to the one constructed in this study can be improved by integrating Kolmogorov-Arnold Network layers instead of linear layers within the MLP architecture (Liu et al., 2024). Kolmogorov-Arnold Networks, or KANs, are alternatives to MLPs publicized for their human interpretability and improved complexity through learnable activation functions, which could allow for more robust classification in lieu of tasks being forced towards regression. The learnable activation functions of KANs would reinforce connections of stability between

molecular structures without introducing biases through thresholds of stability in order to derive more meaning from sparse data of currently available chemical space. The implementation of KANs would also improve rational design over an MLP-based approach, which has limited human interpretability of thresholds of neurons based on activation functions, so creating a framework that reliably learns and fine-tunes the connections between neurons of structure-property relationships could produce complexity of QSPR predictions without introducing volatility. In that same vein of increasing robustness of QSPR modeling, regression model outputs for multivariable DMPK prediction can be explored with a categorical representation contextualization and data augmentation approach through VAEs, which can increase the applicability of the models to predict molecular properties essential to have scalable values of. Finally, sensitivity-specificity analysis could be applied to QSPR prediction to determine performance of models with applications that require heavy insurance of safety for consumers. This integration into industry-friendly and safe QSPR modeling could dovetail well with a GUI and dedicated toolkit for researchers in R&D.

Relation to Prior Literature

Data sparse contextualization approaches to QSPR modeling of metabolic stability of drug-like small molecules have not been explored en masse prior to this study to the knowledge of the author and colleagues, but previous data contextualization approaches to QSAR prediction have been largely insignificant in comparison to this study's results. For one, the aforementioned Eckmann and colleagues' 2024 study explored simple Morgan fingerprint representations as well, and despite the further precedent of QSAR modeling in the field, had a ROC-AUC of approximately 0.58 with meta-learning baselines reaching a maximum of 0.57 ROC-AUC. This indicates that the method detailed in this study has the potential to become the foremost approach for sparse data contextualized QSPR model implementation to improve molecular metabolic stability prediction.

Additionally, QSPR models utilizing classifications akin to those designed in this study without utilizing contextualization through transfer learning have not been able to account for the data sparsity of the metabolic stability QSPR prediction problem (Shah et al., 2016). As such, these papers have outlined data sparsity as a main limitation, which could be remedied by this study and the data augmentation approach outlined previously. For instance, Shah and colleagues' 2016 publication outline that the lack of significance in half-life prediction is due to the paucity of publicly available metabolic stability data.

Conclusion

This research addresses a critical challenge in drug discovery: accurately predicting the metabolic stability of small molecules, a key determinant of drug efficacy and safety. Traditional QSPR models often suffer from limited accuracy due to inadequate molecular representations, the inability to capture categorical dependencies, and most pertinently, the sparsity of metabolic stability data to adequately train a QSPR model on. To overcome these limitations, this study proposes a novel integration of variational autoencoders (VAEs) with categorical representation (CR) contextualization to enhance molecular feature representation. The methodology utilizes Morgan fingerprints as input, and trains a general VAE on 2.4 million molecules and fine tune on a dataset of 2637 molecules, divided into three classes based on metabolic stability: stable, moderately stable, and unstable, as defined in the dataset from Esaki et al.

(2019). The CRs for each class are extracted from the VAE's latent space and concatenated with the molecular fingerprints before being fed into a multilayer perceptron (MLP) classifier. This allows the model to capture both continuous latent features and categorical stability-related information. The classification task was evaluated using ROC-AUC scores to assess the model's ability to differentiate between stable and unstable molecules. The VAE-CR model yielded a ROC-AUC of 0.85, while the baseline model without CR achieved 0.87, indicating no significant improvement in predictive performance. The dataset included an approximately equal distribution across the three classes, but the lack of overlapping molecule screening may have led to data leakage, impacting the reliability of the results. Additionally, the binary nature of Morgan fingerprints could have constrained the model's capacity to capture more nuanced molecular features. Limitations include the binary classification framework, which oversimplifies the spectrum of metabolic stability, and the choice of molecular representations, which may lack richness. Future work should explore richer embeddings, such as BERT-based SMILES representations, and shift to a regression-based approach to more accurately reflect metabolic stability as a continuous variable. Kolmogorov-Arnold Network layers can also be incorporated to improve interpretability and robustness in tandem with VAE-based data augmentation to mitigate the effects of data sparsity. This research sets the foundation for more interpretable and scalable machine learning models in computational-aided drug design, and can be applied in a GUI-based program when in future development to optimize elucidated leads or prevent premature synthesis of unstable leads. By addressing current limitations in molecular representation and model interpretability, this approach has the potential to revolutionize how metabolic stability is predicted, ultimately accelerating the drug development pipeline and reducing the time and cost of bringing new therapies to market.

Appendices

Note: The following code segments are extracted from a context of training loops within the proprietary CADDCore package. As such, none of the training loops, hyperparameter finetuning, or any changes to the CADDCore package that the student researcher made to integrate and accommodate this study's scripts can be shown in the appendices of the paper. The appendices provided show the standalone scripts that the student researcher wrote, then integrated into CADDCore, but do not encompass the entire machine learning process that was enveloped into the CADDCore framework and edited accordingly by the student researcher.

Appendix A: VAE Architecture and Data Conversion

Python

```
#!/usr/bin/env python
# PyTorch imports
import torch
import torch.nn as nn
from utils.plotting_latent import LatentPlotter
from torch.utils.data import Dataset
from transformers import AutoTokenizer, AutoModel
from rdkit import Chem
from rdkit.Chem import AllChem
import numpy as np
# Project imports
from modules.architectures.mlp import MLP
class Encoder(nn.Module):
    """
    Encoder part of the VAE that maps input data to a latent space.
    Attributes
    -----
    fc : MLP
    MLP network for the encoder.
    fc_mean : nn.Linear
    Fully connected layer for the mean of the latent space.
    fc_logvar : nn.Linear
    Fully connected layer for the log variance of the latent space.
    """
    def __init__(self, input_dim, hidden_dims, latent_dim):
        super(Encoder, self).__init__()
        self.fc = MLP(input_dim, hidden_dims[:-1], hidden_dims[-1], layer_normalization=True,
output_activation_function_name='leaky_relu', output_dropout_rate=0.10)
        self.fc_mean = nn.Linear(hidden_dims[-1], latent_dim)
        self.fc_logvar = nn.Linear(hidden_dims[-1], latent_dim)
    def forward(self, x):
        h = self.fc(x)
        z_mean = self.fc_mean(h)
        z_logvar = self.fc_logvar(h)
        return z_mean, z_logvar
class Decoder(nn.Module):
    """
    Decoder part of the VAE that reconstructs input data from the latent space.
    Attributes
    -----
    fc : MLP
    MLP network for the decoder.
    """
    def __init__(self, latent_dim, hidden_dims, output_dim):
        super(Decoder, self).__init__()
```

```

        self.fc = MLP(latent_dim, hidden_dims, output_dim, layer_normalization=True,
output_activation_function_name='leaky_relu')
    def forward(self, z):
        x_reconstructed = self.fc(z)
        return x_reconstructed
class VAE(nn.Module):
    """
    Variational Autoencoder (VAE) that includes an encoder, a decoder, and a reparameterization step.
    Attributes
    -----
    encoder : Encoder
    Encoder network.
    decoder : Decoder
    Decoder network.
    """
    def __init__(self, input_dim, hidden_dims, latent_dim):
        super(VAE, self).__init__()
        self.encoder = Encoder(input_dim, hidden_dims, latent_dim)
        self.decoder = Decoder(latent_dim, hidden_dims[:-1], input_dim)
    def reparameterize(self, z_mean, z_logvar):
        """
        Reparameterization trick to allow backpropagation through the sampling process.
        Parameters
        -----
        z_mean : torch.Tensor
        Mean of the latent space distribution.
        z_logvar : torch.Tensor
        Log variance of the latent space distribution.
        Returns
        -----
        z : torch.Tensor
        Sampled latent variable.
        """
        std = torch.exp(0.5 * z_logvar)
        eps = torch.randn_like(std)
        return z_mean + eps * std
    def forward(self, x):
        """
        Forward pass through the VAE.
        Parameters
        -----
        x : torch.Tensor
        Input tensor.
        Returns
        -----
        x_reconstructed : torch.Tensor
        Reconstructed input tensor.
        z_mean : torch.Tensor
        Mean of the latent space distribution.
        z_logvar : torch.Tensor
        Log variance of the latent space distribution.
        """
        z_mean, z_logvar = self.encoder(x)
        plotter = LatentPlotter(z_mean, z_logvar)
        fig = plotter.plot_latent_space()
        z = self.reparameterize(z_mean, z_logvar)
        x_reconstructed = self.decoder(z)
        return x_reconstructed, z_mean, z_logvar, fig
class MoleculeSMILESDataSet(Dataset):

```

```

def __init__(self, smiles_list, result_label_list=None, transform=None,
             embedding_method="morgan", embedding_dim_morgan=2048,
             pretrained_bert_model="seyonec/ChemBERTa-zinc-base-v1"):
    """
    Initializes the dataset with SMILES strings and optional labels.
    Parameters:
    - smiles_list (list): List of SMILES strings representing the molecules.
    - result_label_list (list, optional): List of labels corresponding to the SMILES strings.
    - transform (callable, optional): Optional transform to be applied on a sample.
    - embedding_method (str): The method to use for embeddings. Options are 'bert', or 'morgan'.
    - embedding_dim_morgan (int): The dimension of the Morgan fingerprint embedding.
    - pretrained_bert_model (str): Path or identifier for the pretrained BERT model.
    """
    with open(smiles_list, 'r') as file:
        self.smiles_list = [line.strip() for line in file]
    self.result_label_list = result_label_list
    self.transform = transform
    # Embeddings
    self.embedding_method = embedding_method
    if self.embedding_method == 'bert':
        self.tokenizer = AutoTokenizer.from_pretrained(pretrained_bert_model)
        self.bert_model = AutoModel.from_pretrained(pretrained_bert_model)
        self.bert_model.eval() # Set the model to evaluation mode to deactivate dropout layers
        self.embedding_dim = self.bert_model.config.hidden_size
    elif self.embedding_method == "morgan":
        self.embedding_dim = embedding_dim_morgan # Default size for Morgan fingerprints
    else:
        raise ValueError("Unsupported embedding method: {}".format(self.embedding_method))
    # Result labels
    if result_label_list is not None:
        if isinstance(result_label_list, list) and len(result_label_list) == len(self.smiles_list):
            self.result_size = len(result_label_list[0]) if isinstance(result_label_list[0], list) else 1
        elif isinstance(result_label_list, str):
            with open(result_label_list, 'r') as file:
                lines = file.readlines()
                self.result_label_list = [list(map(float, line.strip().split(','))) for line in lines]
                assert len(self.result_label_list) == len(self.smiles_list), "Labels list must match SMILES list
length."
            self.result_size = len(self.result_label_list[0])
        else:
            raise ValueError("result_label_list should be either a list of labels or a path to a file containing
labels.")
    else:
        print("No results labels provided for MoleculeSMILESDataSet; "
              "result labels will be feature embeddings (i.e., reconstruction task)")
    self.result_size = self.embedding_dim

    def __len__(self):
        return len(self.smiles_list)

    def __getitem__(self, idx):
        """
        Retrieves an item by index.
        Parameters:
        - idx (int): Index of the item to retrieve.
        Returns:
        - sample (dict): Dictionary containing the features and, optionally, the label.
        """
        # Some exception handling is required when working with SMILES and RDKit

```

```

while True:
    try:
        smiles = self.smiles_list[idx]

        if self.embedding_method == "bert":
            inputs = self.tokenizer(smiles, return_tensors="pt", padding=True, truncation=True)
            with torch.no_grad():
                outputs = self.bert_model(**inputs)
            # embeddings = outputs.last_hidden_state.squeeze(0) # Remove the batch dimension
            embeddings = outputs.last_hidden_state.mean(dim=1).squeeze(0) # Average over sequence length
        elif self.embedding_method == "morgan":
            mol = Chem.MolFromSmiles(smiles)
            embeddings = AllChem.GetMorganFingerprintAsBitVect(mol, radius=2, nBits=2048)
            embeddings = torch.tensor(np.array(embeddings), dtype=torch.float32)
            sample = {'features': embeddings}
            if self.result_label_list is not None:
                result_label = self.result_label_list[idx]
                sample['results'] = torch.tensor(result_label, dtype=torch.float) # Prediction
            else:
                sample['results'] = embeddings # Reconstruction
            if self.transform:
                sample = self.transform(sample)
            return sample
    except Exception as e:
        print(f"Skipping invalid molecule at index {idx}: {e}")
        idx += 1
        if idx >= len(self.smiles_list):
            raise IndexError("No valid molecules found in the dataset!")

def num_features(self):
    """
    Get the number of feature labels in the dataset.
    Returns:
    int: Number of features.
    """
    # Retrieve the embedding size from the model configuration
    return self.embedding_dim

def num_feature_columns(self):
    return self.num_features()

def num_results(self):
    """
    Get the number of result labels in the dataset.
    Returns:
    int: Number of results.
    """
    return self.result_size

```

Appendix B: Latent Space Visualization Excerpt from train_model script

Python

```

elif self.model_type == "vae":
    features = batch["features"].to(self.device, non_blocking=True)
    predictions, z_mean, z_logvar, fig = self.model(features)
    predictions.to(self.device, non_blocking=True)
    z_mean.to(self.device, non_blocking=True)
    z_logvar.to(self.device, non_blocking=True)
    buf = BytesIO()

```

```

plt.savefig(buf)
buf.seek(0)
im = Image.open(buf)
latent_frames.append(im)
results = batch["results"].to(self.device, non_blocking=True)
loss = self.loss_module(predictions, results, z_mean, z_logvar)

```

Appendix C: Steps 3-5 Context Generation and Appendation

Python

```

#!/usr/bin/env python
import torch
from modules.architectures.vae import VAE
import numpy as np
import pandas as pd
import h5py
import os
def main():
    latent_dim = 128
    stable_dict =
torch.load("/home/ahdoutj/Projects/dmpk_vae/finetune/stable/smiles.stablevae finetune.model.h5")
    stable_model = VAE(2048, [1024, 512, 256, 128], 128)
    stable_model.load_state_dict(stable_dict)
    stable = stable_model.decoder
    stable.eval()
    mod_dict =
torch.load("/home/ahdoutj/Projects/dmpk_vae/finetune/moderate/smiles.moderatevae finetune.model.h5")
    mod_model = VAE(2048, [1024, 512, 256, 128], 128)
    mod_model.load_state_dict(mod_dict)
    moderate = mod_model.decoder
    moderate.eval()
    unstable_dict =
torch.load("/home/ahdoutj/Projects/dmpk_vae/finetune/unstable/smiles.unstablevae finetune.model.h5")
    unstable_model = VAE(2048, [1024, 512, 256, 128], 128)
    unstable_model.load_state_dict(unstable_dict)
    unstable = unstable_model.decoder
    unstable.eval()
    with torch.no_grad():
        for i in range(2637):
            random_latent_sample = torch.randn(latent_dim)
            augmented_stable = torch.cat(
                (augmented_stable, torch.reshape(torch.tensor(stable(random_latent_sample)), (1, 2048))),
                0) if i != 0 else torch.reshape(torch.tensor(stable(random_latent_sample)), (1, 2048))
        for i in range(2637):
            random_latent_sample = torch.randn(latent_dim)
            augmented_moderate = torch.cat(
                (augmented_moderate, torch.reshape(torch.tensor(moderate(random_latent_sample)), (1,
2048))),
                0) if i != 0 else torch.reshape(torch.tensor(moderate(random_latent_sample)), (1, 2048))
        for i in range(2637):
            random_latent_sample = torch.randn(latent_dim)

```

```

        augmented_unstable = torch.cat(
            (augmented_unstable, torch.reshape(torch.tensor(unstable(random_latent_sample)), (1,
2048))),
            0) if i != 0 else torch.reshape(torch.tensor(unstable(random_latent_sample)), (1, 2048))
    # Save to CSVs
    df_stable = pd.DataFrame(augmented_stable)
    df_stable.to_csv("/home/ahdoutj/Projects/dmpk_vae/final_predictor/stable_cr.csv", header=False,
index=False)
    df_moderate = pd.DataFrame(augmented_moderate)
    df_moderate.to_csv("/home/ahdoutj/Projects/dmpk_vae/final_predictor/moderate_cr.csv",
header=False, index=False)
    df_unstable = pd.DataFrame(augmented_unstable)
    df_unstable.to_csv("/home/ahdoutj/Projects/dmpk_vae/final_predictor/unstable_cr.csv",
header=False, index=False)
    global s_emb, m_emb, u_emb
    stable_cr = torch.tensor(np.array(df_stable))
    moderate_cr = torch.tensor(np.array(df_moderate))
    unstable_cr = torch.tensor(np.array(df_unstable))
    stable_emb = h5py.File("/home/ahdoutj/Projects/dmpk_vae/final_predictor/stable_embeddings.h5",
'r')
    for dset in stable_emb['features'].keys():
        s_emb = np.append(s_emb, stable_emb['features'][dset][:], 0) if float(dset) != 0 else
stable_emb['features']['0'][:]:]
        stable_emb = torch.tensor(s_emb)
        moderate_emb =
h5py.File("/home/ahdoutj/Projects/dmpk_vae/final_predictor/moderate_embeddings.h5", 'r')
        for dset in moderate_emb['features'].keys():
            m_emb = np.append(m_emb, moderate_emb['features'][dset][:], 0) if float(dset) != 0 else
moderate_emb['features']['0'][:]:]
            moderate_emb = torch.tensor(m_emb)
            unstable_emb =
h5py.File("/home/ahdoutj/Projects/dmpk_vae/final_predictor/unstable_embeddings.h5", 'r')
            for dset in unstable_emb['features'].keys():
                u_emb = np.append(u_emb, unstable_emb['features'][dset][:], 0) if float(dset) != 0 else
unstable_emb['features']['0'][:]:]
                unstable_emb = torch.tensor(u_emb)
            result_labels = torch.cat((torch.tensor([1]*1138), torch.tensor([0]*1499)), 0)
            result_labels = torch.reshape(result_labels, (2637, 1))
            embeddings = torch.cat((stable_emb, moderate_emb, unstable_emb), 0)
            crs = torch.cat((stable_cr, moderate_cr, unstable_cr), 1)
            labelled_final_nocr = pd.DataFrame(torch.cat((embeddings, result_labels), 1))
            labelled_final_nocr = labelled_final_nocr.sample(frac=1)
            labelled_final_nocr.to_csv("/home/ahdoutj/Projects/dmpk_vae/final_predictor/concatenated_data.nocrs.csv",
header=False, index=False)
            labelled_final = pd.DataFrame(torch.cat((crs, torch.tensor(np.array(labelled_final_nocr))), 1))
            labelled_final.to_csv("/home/ahdoutj/Projects/dmpk_vae/final_predictor/concatenated_data.crs.csv",
header=False,
                                index=False)
if __name__ == "__main__":
    main()

```

References

- Chen, M.; Borlak, J.; Tong, W. High lipophilicity and high permeability are associated with hepatotoxicity of drugs. *Computational and Structural Biotechnology Journal* 2012, 12 (13), 10-16.
- Cherkasov, A.; Muratov, E. N.; Fourches, D.; Varnek, A.; Baskin, I. I.; Cronin, M.; ... Tropsha, A. QSAR modeling: where have you been? Where are you going to? *Journal of Medicinal Chemistry* 2014, 57 (12), 4977-5010.
- Eckmann, P.; Anderson, J.; Yu, R.; Gilson, M. K. (2024). Ligand-Based Compound Activity Prediction via Few-Shot Learning. *Journal of chemical information and modeling*, 64(14), 5492–5499. <https://doi.org/10.1021/acs.jcim.4c00485>
- Esaki, T.; Watanabe, R.; Kawashima, H.; Ohashi, R.; Natsume-Kitatani, Y.; Nagao, C.; Mizuguchi, K. (2019). Data Curation can Improve the Prediction Accuracy of Metabolic Intrinsic Clearance. *Molecular informatics*, 38(1-2), e1800086. <https://doi.org/10.1002/minf.201800086>
- Gasteiger, J.; Engel, T. *Cheminformatics: A Textbook*; Wiley-VCH: Weinheim, Germany, 2003.
- Gómez-Bombarelli, R.; Duvenaud, D.; Hernández-Lobato, J.; Aguilera-Iparraguirre, J.; Hirzel, T.; Adams, R.; Aspuru-Guzik, A. (2016). Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules. *ACS Central Science*. 4. 10.1021/acscentsci.7b00572.
- Gunaydin, H.; Altman, M. D.; Ellis, J. M.; Fuller, P.; Johnson, S. A.; Lahue, B.; Lapointe, B. (2018). Strategy for Extending Half-life in Drug Design and Its Significance. *ACS medicinal chemistry letters*, 9(6), 528–533. <https://doi.org/10.1021/acsmmedchemlett.8b00018>
- Johnson, R. P. Advances in QSAR modeling. *Chemical Reviews* 2013, 113 (7), 4639-4702.
- Jones, C. A. H.; Brown, B. P.; Schultz, D. C.; Engers, J.; Kramlinger, V. M.; Meiler, J.; Lindsley, C. W. *ACS Chemical Neuroscience* 2024 15 (12), 2396-2407. DOI: 10.1021/acscchemneuro.4c00086
- Kerns, E. H.; Di, L. *Drug-like properties: Concepts, structure design and methods*; Elsevier: New York, 2008.
- Liu, Z.; Wang, Y.; Vaidya, S.; Ruehle, F.; Halverson, J.; Soljačić, M.; Hou, T. Y.; Tegmark, M. (2024). KAN: Kolmogorov-Arnold Networks. <https://doi.org/10.48550/arXiv.2404.19756>

- Lovrić, M.; Đuričić, T.; Tran, H. T. N.; Hussain, H.; Lacić, E.; Rasmussen, M. A.; Kern, R. (2021). Should We Embed in Chemistry? A Comparison of Unsupervised Transfer Learning with PCA, UMAP, and VAE on Molecular Fingerprints. *Pharmaceuticals (Basel, Switzerland)*, 14(8), 758. <https://doi.org/10.3390/ph14080758>
- Medina-Franco, JL (2021). Grand Challenges of Computer-Aided Drug Design: The Road Ahead. *Front. Drug. Discov.* 1:728551. doi: 10.3389/fddsv.2021.728551
- Muenkler, H.; Misztela, H.; Pikusa, M.; Segler, M.; Schneider, N.; Maziarz, K. (2023). Are VAEs Bad at Reconstructing Molecular Graphs?. 10.48550/arXiv.2305.03041.
- Ochiai, T.; Inukai, T.; Akiyama, M.; Furui, K.; Ohue, M.; Matsumori, N.; Inuki, S.; Uesugi, M.; Sunazuka, T.; Kikuchi, K.; Takeya, H.; Sakakibara, Y. (2023). Variational autoencoder-based chemical latent space for large molecular structures with 3D complexity. *Communications chemistry*, 6(1), 249. <https://doi.org/10.1038/s42004-023-01054-6>
- Pan, S. J.; Yang, Q. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering* 2010, 22 (10), 1345-1359.
- Rakhimbekova, A.; Madzhidov, T. I.; Nugmanov, R. I.; Gimadiev, T. R.; Baskin, I. I.; Varnek, A. (2020). Comprehensive Analysis of Applicability Domains of QSPR Models for Chemical Reactions. *International journal of molecular sciences*, 21(15), 5542. <https://doi.org/10.3390/ijms21155542>
- Shah, P.; Kerns, E.; Nguyen, D. T.; Obach, R. S.; Wang, A. Q.; Zakharov, A.; McKew, J.; Simeonov, A.; Hop, C. E.; & Xu, X. (2016). An Automated High-Throughput Metabolic Stability Assay Using an Integrated High-Resolution Accurate Mass Method and Automated Data Analysis Software. *Drug metabolism and disposition: the biological fate of chemicals*, 44(10), 1653–1661. <https://doi.org/10.1124/dmd.116.072017>
- Simonovsky, M.; Komodakis, N. (2018). GraphVAE: Towards Generation of Small Graphs Using Variational Autoencoders. 10.48550/arXiv.1802.03480.
- Smith, J. M.; Jones, A. D. Impact of metabolic stability on drug design. *Drug Metabolism Reviews* 2011, 43 (3), 391-408.
- Subramaniam, S.; Mehrotra, M.; & Gupta, D. (2008). Virtual high throughput screening (vHTS)--a perspective. *Bioinformatics*, 3(1), 14–17. <https://doi.org/10.6026/97320630003014>

- Torres, L.H.; Arrais, J.P.; & Ribeiro, B. (2023). Few-shot learning via graph embeddings with convolutional networks for low-data molecular property prediction. *Neural Computing and Applications*, 35, 13167-13185.
- Weininger, D. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences* 1988, 28 (1), 31-36.